

Here's the complete walkthrough, assuming you're starting from zero.

Step 1: Extract the project

Download stage1-app.zip, unzip it somewhere on your computer (like Desktop or a Projects folder). You'll get a stage1-app folder with backend/ and frontend/ inside it.

Step 2: Install the tools you need

Check what you already have by opening a terminal and running:

```
bash
```

```
python3 --version # need 3.11+
```

```
node --version    # need 18+
```

If either is missing, install:

- **Python:** python.org/downloads
- **Node.js:** nodejs.org (get the LTS version)
- **Docker Desktop** (easiest way to run Redis): docker.com/products/docker-desktop

Step 3: Create your MongoDB Atlas account (free)

1. Go to mongodb.com/cloud/atlas/register and sign up.
2. Create a free cluster (choose the "M0 Free" tier).
3. When prompted, create a **database user** — set a username and password, and **write the password down**.
4. Under **Network Access**, click "Add IP Address" → choose "Allow Access from Anywhere" (0.0.0.0/0) — fine for local development.
5. Once the cluster is ready, click **Connect → Drivers**, choose Python, and copy the connection string. It looks like:

```
mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority
```

6. Replace <username> and <password> with the ones you created in step 3.

Step 4: Start Redis

With Docker Desktop open and running, in any terminal:

```
bash
```

```
docker run -d --name redis-stage1 -p 6379:6379 redis:7-alpine
```

Check it's running:

```
bash
```

```
docker ps
```

You should see redis-stage1 in the list.

Step 5: Set up the backend (Django)

Open a terminal, navigate into the project:

```
bash
```

```
cd path/to/stage1-app/backend
```

Create and activate a virtual environment:

```
bash
```

```
python3 -m venv venv
```

```
source venv/bin/activate    # Mac/Linux
```

```
venv\Scripts\activate     # Windows
```

Your terminal prompt should now show (venv) at the start.

Install dependencies:

```
bash
```

```
pip install -r requirements.txt
```

Create your environment file:

```
bash
```

```
cp .env.example .env
```

Open .env in a text editor and paste your MongoDB Atlas connection string into MONGO_URI. Leave everything else as-is.

Run Django's initial setup and start the server:

```
bash
```

```
python manage.py migrate
```

```
python manage.py runserver 0.0.0.0:8000
```

Leave this terminal running.

Test it worked — open a new terminal (don't close the running one) and run:

```
bash
```

```
curl http://localhost:8000/api/health/
```

You should see:

```
json
```

```
{"api": "ok", "mongodb": "ok", "redis": "ok"}
```

If mongodb shows an error, double-check your password in the connection string (special characters may need URL-encoding). If redis shows an error, re-check docker ps.

Step 6: Set up the frontend (React)

Open a **new, separate terminal** (keep Django running in the other one):

```
bash
```

```
cd path/to/stage1-app/frontend
```

```
npm install
```

```
cp .env.example .env
```

```
npm run dev
```

It'll print something like Local: <http://localhost:5173/>.

Step 7: Open the app

Go to <http://localhost:5173> in your browser. You should see the Task Board — type a task, click Add, and it should appear in the list.

Quick summary of what's running

Terminal What's running

1 Django backend (localhost:8000)

2 React frontend (localhost:5173)

Docker Redis container in background

Cloud MongoDB Atlas (nothing to run locally)

You need terminals 1 and 2 both running at the same time for the app to work. Let me know if you hit an error at any step and paste the exact message — happy to debug it with you.

now tell me what does redis doing in the above application and how to access redis and check

Orchestrated Redis's role and mapped verification methods comprehensively

Orchestrated Redis's role and mapped verification methods comprehensively

What Redis is doing in this app

Redis sits between the Django API and MongoDB Atlas as a **cache** for the task list. Here's the exact flow, from `tasks/views.py` and `tasks/redis_client.py`:

On GET /api/tasks/:

1. Django first checks Redis for a key called `tasks:list`.
2. **If found (cache hit):** it returns that cached JSON immediately — no MongoDB query at all. Response shows `"source": "cache"`.
3. **If not found (cache miss):** it queries MongoDB Atlas for all tasks, sends them back, and *also* stores that result in Redis with a 30-second expiry (`REDIS_CACHE_TTL_SECONDS` in `.env`). Response shows `"source": "database"`.

On POST (create), PUT (update), or DELETE:

- After writing to MongoDB, Django immediately deletes the tasks:list key from Redis (invalidate_task_list_cache()). This forces the *next* GET to be a cache miss, so it goes back to MongoDB and rebuilds the cache with fresh data — otherwise you'd see stale data for up to 30 seconds after making a change.

Why bother? In a real app with many users hitting GET /api/tasks/ repeatedly, this avoids hammering MongoDB Atlas for the same unchanged data every single time — Redis serves it from memory instead, which is much faster and cheaper. In this small demo you won't feel a speed difference, but the pattern is the important part.

You can literally watch this happen in the UI: load the page (badge says "database"), refresh within 30 seconds (badge says "cache"), wait 30+ seconds and refresh again (back to "database").

How to access and inspect Redis directly

Since Redis is running in a Docker container, use docker exec to get into its CLI:

```
bash
```

```
docker exec -it redis-stage1 redis-cli
```

You'll get a Redis prompt like 127.0.0.1:6379>. Try these commands:

See all keys currently stored:

```
KEYS *
```

After you've loaded the task list at least once in the browser, you should see:

```
1) "tasks:list"
```

View the cached value:

```
GET tasks:list
```

This prints the raw JSON string of tasks currently cached.

Check how much time is left before it expires:

```
TTL tasks:list
```

Returns seconds remaining (counts down from 30, or -2 if the key doesn't exist / already expired).

Manually delete the cache (simulate invalidation):

```
DEL tasks:list
```

Confirm Redis is alive:

```
PING
```

Should reply PONG.

Exit the CLI:

```
exit
```

Quick check without going into Redis directly

You already have this built in — hit the health endpoint:

```
bash
```

```
curl http://localhost:8000/api/health/
```

The "redis": "ok" field confirms Django can successfully PING Redis. If Redis were down, it'd show "redis": "error: ..." instead, and the app would still work — it'd just always hit MongoDB directly since caching fails silently (that's intentional, see the try/except in redis_client.py — a cache outage shouldn't take the whole app down).